

목진희 · AI 에이전트 엔지니어

jinheemok815@gmail.com · GitHub Jin-tonix · 포트폴리오 jin-tonix.github.io

StyleSeller 에서 사내 업무 웹·CS 에이전트·지식 허브·직원 PC 배포를 설계부터 운영까지 혼자 맡고 있습니다. 비전공 전환자이고, 개발은 2026.02 에 시작해 곧 2년차입니다.

경력

StyleSeller (SNS 인플루언서 공동구매 플랫폼) · 2025.12 ~ 현재

- 직책: 개발자 (AI·자동화 전담·책임). 개발 업무는 2026.02 부터입니다.
- 담당: 사내 AI 업무 시스템의 설계·개발·운영 1인. 사용자는 사내 직원입니다.
- 범위: 저장소 4개(brand-tool · agent-company · email-agent · kakaocli)가 한 운영 시스템으로 맞물립니다.

We:Review (Welead) · 인턴 · 2025.08 ~ 2025.10

- OCR + LLM 영수증 검수 파이프라인을 혼자 구축했습니다.

프로젝트

1. 벤더 CS 에이전트 — 공식 API 없는 카톡에서 도는 거래처 CS

2026.06 ~ 현재 · 설계·개발·운영 1인 100% · Python · FastAPI · Swift(macOS) · LiteLLM → Gemini 2.5 Flash · LightRAG · MCP · Chrome MV3

- **문제:** 거래처(브랜드·판매 파트너사) 문의가 직원 개인 카톡으로 흩어져 들어왔습니다. 카톡 개인톡에는 공식 API 가 없어 회사가 기록을 한곳에서 볼 수 없었습니다.
- **한 일**
- 직원 맥(kakaocli 포크)·윈도우 수집기로 대화를 모아 거래처별 원장·시트·드라이브에 기록했습니다. 서버가 받았다고 응답한 뒤에만 체크포인트를 옮깁니다.
- 사내 지식 허브를 도구 호출 루프로 검색해 근거를 붙여 답하게 했습니다. 회사 전체를 조회하는 Spark 도구는 이 챗봇에서 막았습니다(fail-closed).
- 관문을 통과한 답은 발송 잡이 되어 직원 맥 로컬 발송기가 kakaocli 로 보내고, 도착까지 확인합니다. 테스트 방 밖의 실제 거래처 응대는 직원 패널(크롬 확장)에서 사람이 고쳐 같은 발송기로 보냅니다.
- 다른 거래처 정보는 세 겹으로 막았습니다(워크스페이스 분리 · 도구 단계 치환 · temperature 0 LLM 최종 판정, 실패하면 닫힘).
- 자동 답변은 코드 기본값인 테스트 방 스위치로 묶었습니다. 판정할 수 없으면 발송만 막고 초안은 남깁니다.
- **결과**
- 수집·기록 운영 중, 요약은 소수 수신자 대상. 자동 답변은 2026-08~09 중순 실제 거래처 방에서 돌린 뒤(09-01 부터 봇을 태그한 문의에만), 2026-09-15 부터 개선판을 테스트 방에서 검증 중입니다.
- 벤더 에이전트(수집·기록·게이트) 61,437줄 · 테스트 파일 107개 (agent-company, 2026-09-23, 챗봇 서비스 제외).

2. SS 업무툴 (brand-tool) — 엑셀 가격표에서 직원이 쓰는 사내 업무 웹으로

2026.03 ~ 현재 · 설계·개발·운영 1인 100% · Next.js 16 · React 19 · TypeScript · Postgres + PostgREST · Vertex AI Gemini 2.5 · n8n · Playwright

- **문제:** 영업 직원이 제안서마다 구글 시트 가격표로 파트너·셀러 몫을 손으로 계산했습니다. 상품 데이터는 시트와 홈페이지 DB 에 흩어져 있었습니다.
- **한 일**
- 가격 수식을 코드로 옮기고 실무자와 한 칸씩 대조했습니다(계산 테스트 43케이스). LLM 은 고칠 필드만 JSON 으로 돌려주고, 금액은 코드가 다시 계산합니다.
- 공급사 카탈로그(PDF·이미지·엑셀)를 Gemini 로 읽어 상품·옵션·가격으로 만드는 입력 화면을 만들었습니다.
- 판매 파트너사 매칭을 규칙 점수 U 벡터 검색 후보 + LLM 판정으로 설계했습니다. 제안 카톡은 판매 파트너사당 하루 오전·오후 1회로 서버가 막습니다.
- 셀러찾기를 직원 PC 위커로 옮기고(한 인스타 계정은 한 PC), 맞춤카톡은 500명 이상이면 승인 대기로 들어가게 했습니다.
- 업무 웹은 벤더 CS 에이전트와 Spark 가 부르는 업무 데이터 백엔드(/api/spark/* 14개)도 겸합니다.
- **결과**
- 직원이 로그인해 쓰는 라이브 화면 14개(로그인·상세 화면 포함, 2026-09-23). 맞춤카톡 2026-09-22 하루 261명 발송. 셀러찾기 위커 PC 9대(2026-09-18).
- 커밋 1,458개 · API 라우트 134개 · 테스트 파일 205개 (main, 2026-09-23).

3. Spark · Work OS — 에이전트가 안전하게 실행되는 사내 운영 플랫폼

2026.05 ~ 현재 · 설계·개발·운영 1인 100% · TypeScript · Express · MCP · LiteLLM · Claude Code CLI · Postgres · Paperclip(오픈소스 포크)

- **문제:** 업무 데이터가 DB 4개에 나뉘어 있었고, 메일 에이전트·DM 봇·n8n 이 따로 움직였습니다. 2026-05-20 기준 에이전트 플랫폼에 등록된 에이전트는 0개였습니다.
- **한 일**
- 직원이 업무 DB 4개를 말로 묻고 출처가 붙은 답을 받는 Spark 를 업무 웹 상단 메뉴 첫 칸에 붙였습니다. DB 는 읽기 전용 역할로만 조회합니다.
- 요청 본문의 역할을 믿지 않고 서버가 DB 에서 다시 찾게 했습니다.

- Tool Registry 실행 게이트를 만들었습니다. 기본은 드라이러이고, 실행 직전 미등록·킬 스위치·역할·승인 행 DB 재검증·역등 키를 확인합니다. 지금 승인 뒤 실제로 실행되는 것은 이슈 생성·도구 멈춤이고, 외부 발송 도구는 등록·기록 단계입니다.
- 첫 계획의 Temporal·LangGraph 는 넣지 않았습니다. 에이전트가 0개일 때는 운영 비용만 늘기 때문입니다.
- **결과**
- Tool Registry 도구 20개 등록, 매일 운영에 쓰이는 에이전트 5개 (org/release, 2026-09-23).
- 새로 만든 파일 120개 · 27,506줄(빈 줄·테스트 12,880줄 포함, 같은 기준).

4. 메일 에이전트 · 사내 지식 허브 — 근거를 강제하는 답장 초안

2026.03 ~ 현재 · 설계·개발·운영 1인 100% · Python · FastAPI · LightRAG · Neo4j · pgvector · Gemini 2.5 Flash · n8n · Gmail API

- **문제:** 입점·공구·제휴 메일 답장에 회사 사실이 정확히 들어가야 했습니다. 평면 벡터 검색으로는 회사·담당자·과거 요청의 관계를 잊지 못했습니다.
- **한 일**
- LightRAG(Neo4j + pgvector) 지식 그래프로 검색하고, 초안의 문장마다 규칙 검증기·LLM critic 으로 근거를 검사해 다시 쓰게 했습니다.
- 외부 자동 발송은 두지 않았습니다. 브랜드·오더 공용 메일함은 담당자 Gmail 에 초안을 두고 직원이 고쳐 보냅니다. 대표 메일함은 Slack 알림·발송 버튼을 썼고, 2026-09-02 에 이 초안 단계를 멈췄습니다.
- 보낸 메일을 연속 커서로 스캔해 교정 문서로 학습합니다. 배포 중 시간창이 빠지던 문제를 커서로 고쳤습니다.
- **결과**
- 워크스페이스 13개를 벤더 CS·Spark·판매 파트너사 매칭이 함께 씁니다. 브랜드·오더 공용 메일함 초안은 운영 중이고, 대표 메일함 자동 초안만 2026-09-02 운영 판단으로 멈췄습니다.
- 제품 코드 44,877줄 + 테스트 28,254줄, 커밋 1,418개 (마지막 커밋 2026-09-02 기준).

5. 직원 PC 배포 · 운영 인프라 — 서버가 못 하는 일을 직원 PC 에서

2026.02 ~ 현재 · 설계·개발·운영 1인 100% · Node.js · Chrome MV3 · Swift · PowerShell · launchd · Docker Compose/Swarm · GCP VM · nginx

- **문제:** 인스타 DM·개인 카톡은 공식 API 가 없고, 인스타는 데이터센터 IP 를 막습니다. 일할 수 있는 곳은 직원의 맥·윈도우 PC 였습니다.
- **한 일**
- 한 줄 설치(맥 bash · 윈도우 PowerShell), 도구별 주기의 자동 갱신(zip sha256), PC 별로 발급·취소하는 기기 키를 만들었습니다. PC 에 DB 키를 두지 않습니다.
- 윈도우 카톡 수집을 프로세스 메모리 읽기 전용 스캔으로 다시 설계했습니다(2026-09-15).
- 배포는 커밋한 이미지로만 합니다. CI 이미지가 없으면 :latest 로 물러나지 않고 실패합니다.
- 클라우드 DB 가 전송량 한도로 멈춘 날(2026-08-20), 앱 코드를 그대로 둔 채 VM 자체 호스팅으로 당일 이관·복구했습니다.
- **결과**
- 직원 PC 11대 등록 (카톡 동기화 등록 기준, 맥 6 · 윈도우 5, 2026-09-16). 서버·DB 는 GCP VM 한 대, 운영 컨테이너 39개 (2026-09-23).

AI 활용 방식과 결과

코드는 Claude Code 로 작성했습니다. 아키텍처·규칙·검증·롤아웃은 제가 판단했습니다. Claude 공동작성 트레일러가 있는 커밋은 agent-company 3,646 · brand-tool 1,122 · email-agent 988 개입니다(merge 제외, agent-company org/release-brand-tool main 2026-09-23, email-agent 2026-09-02 기준). 네 저장소의 본인 커밋은 합 7,276개입니다(merge 제외, 2026-09-23 집계). 에이전트가 틀린다는 전제로 설계 문서 → 실행 계획 → 재현 테스트 → 교차 리뷰 → 실기기 확인 → 도는 코드 해시로 배포 확인 순서를 지킵니다. 규칙 파일과 pre-commit 혹은 반복 실수를 커밋 단계에서 막습니다. pre-push 게이트는 되돌리기 등록이 없는 서비스 push 를 막고, 저장소마다 한 번 설치해야 걸리는 속입니다.

- **코드를 부르지 않는 테스트 (2026-09-22):** 벤더 CS 카드가 안 닫히는 버그를 고치다가, 그 배선을 지키던 테스트가 손으로 만든 dict 만 보고 실제 함수를 부르지 않는 것을 찾았습니다. 실제 코드를 태우는 테스트와 메시지 처리 경로 통합 테스트로 바꿨습니다.
- **규칙이 판단을 가로챈 누출 방어 (2026-08-13 ~ 08-30):** 다른 거래처 조건을 정책처럼 답한 사고 뒤 정규식 게이트를 넣었지만, 정규식이 차단까지 해 LLM 판정을 건너뛰었습니다. 정규식은 트리거로만 두고, 최종 판정은 temperature 0 · 5초 · 실패하면 닫히는 LLM 판정으로 바꿨습니다.

이전 경력

- 청주대 법학 전공 뒤 어린이집 운영실장(2015.03–2018.02), 영어 강사(2018.08–2024.02, JLS · Pots&Pans · 몬테키즈)로 일했습니다.
- 수업과 운영 절차를 설계하던 경험을, 업무 흐름에서 규칙으로 돌 부분과 사람이 판단할 부분을 나누는 일에 씁니다.

기술 스택

- **실무 (StyleSeller, 2026.02~):** Python · FastAPI · TypeScript · Next.js 16 · React 19 · Node.js/Express · PostgreSQL/PostgREST · LightRAG · Neo4j · pgvector · LiteLLM(Gemini) · MCP 서버 4개 직접 개발 · Tool Registry(실행 게이트) · Chrome Extension MV3 · Playwright/CDP · Swift(macOS) · PowerShell · Docker Compose/Swarm · GitHub Actions · GCP VM · nginx · n8n · Claude Code
- **교육 과정 (2024):** Java · Spring Boot · MySQL · JPA · Vue.js · Flutter · LangChain · RAG

교육

- 한국IT비즈니스진흥협회 · 생성형 AI 개발자 맞춤형 인재 양성 과정 (2025.05–2025.07)
- 하이라미디아카데미 · AI 과정 (2024.10–2024.12) · 풀스택 과정 (2024.04–2024.10)
- 학점은행제 · 경영학 (2021.02–2023.06)
- 청주대학교 · 법학 학사 (2008.03–2012.02)